



Tecnologias de transmissão de dados em sistemas web

Você verá os conceitos de transmissão e consumo de dados em sistemas web, utilizando os formatos XML, JSON e YAML. Esses conceitos são fundamentais para a integração e transmissão de dados entre sistemas. Nesse sentido, trataremos a utilização de tais formatos de dados nos sistemas web.

Prof. Alexandre de Oliveira Paixão

Preparação

Faça o [download](#) dos códigos e use editores de texto compatíveis com linguagens de marcação, como Notepad++ para Windows e Nano para Linux. Para visualizar os exemplos de código com AJAX, é necessário hospedar as páginas HTML em um servidor web. No Windows, o XAMP com o servidor web Apache é recomendado, ou você pode instalar o Apache ou o IIS separadamente. No Linux, os servidores Apache ou Nginx são recomendados.

Objetivos

- Descrever a linguagem de marcação XML e sua aplicabilidade em sistemas web.
- Descrever os formatos de transmissão de dados JSON e YAML.
- Identificar a forma de utilização de dados nos formatos XML, JSON e YAML em requisições AJAX.

Introdução

Prepare-se para mergulhar nas tecnologias de transmissão de dados em sistemas web! Neste conteúdo, será abordada a linguagem de marcação XML e sua aplicabilidade, além dos formatos de transmissão de dados JSON e YAML. Você compreenderá como utilizar dados nos formatos XML, JSON e YAML em requisições AJAX, explorando estrutura, semelhanças e diferenças. Assista ao vídeo e aprenda a aproveitar essas tecnologias para criar aplicações web eficientes e interativas.

Introdução



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que é a linguagem XML?

A linguagem XML — acrônimo para eXtensible Markup Language — é, a exemplo da HTML, uma linguagem de marcação. Criada pelo W3C (World Wide Web Consortium), em 1996, e transformada em uma recomendação pelo mesmo órgão em 1998, essa linguagem possui algumas importantes diferenças em relação à HTML, visto que foi criada justamente para ser diferente desta — em outras palavras, para estender as funcionalidades da HTML.

Neste vídeo, mergulhe na linguagem de marcação XML e sua aplicabilidade em sistemas web. Descubra como XML difere da HTML, sua estrutura de tags definidas pelo usuário e como criar um arquivo XML bem formatado.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Linguagem XML

É um padrão para a formatação e transmissão de dados de fácil entendimento tanto para humanos quanto para máquinas. Sua principal característica — e diferença para HTML — é ser composta por tags definidas pelo usuário (ou programador). Diferentemente da HTML, na qual todas as tags são predefinidas, em XML, nós mesmos criamos nossas tags.



Atenção

A principal função e utilização da XML é transmitir dados por meio da web.

Anatomia de um arquivo XML

Os documentos XML são constituídos por unidades de armazenamento chamadas entidades, que contêm dados. Esses dados são compostos por caracteres, alguns dos quais formam dados de caracteres enquanto outros formam a marcação (W3C, 2020).

Vejamos, a seguir, nosso primeiro arquivo XML. A partir desse arquivo simples, começaremos a entender o seu formato.

python

```
        física<
José Maria
000.000-00

        jurídica
José Maria SA
111.111-11
```

Em termos de sintaxe, observe os seguintes itens:

Primeira linha

Nesta linha, temos a declaração XML, responsável por especificar a versão e por informar ao navegador que se trata de um arquivo XML. Aqui também é definido o *charset* (conjunto de caracteres utilizados para escrever o documento) do documento.

Segunda linha

Nesta linha, representado pela tag `<correntistas>`, temos o nó principal — ou nó raiz — do documento.

Terceira linha

Nesta linha, temos um comentário. Repare que os comentários em um documento XML são iguais aos usados em HTML.

Quarta linha

Nesta linha, temos o primeiro elemento filho do nó raiz, o elemento `< Pessoa >`.

Quinta, sexta e sétima linhas

Nestas linhas, temos os elementos filhos do elemento `< Pessoa >`.

Linhas seguintes

Nestas linhas, temos outro elemento `< Pessoa >` e seus respectivos elementos filhos.

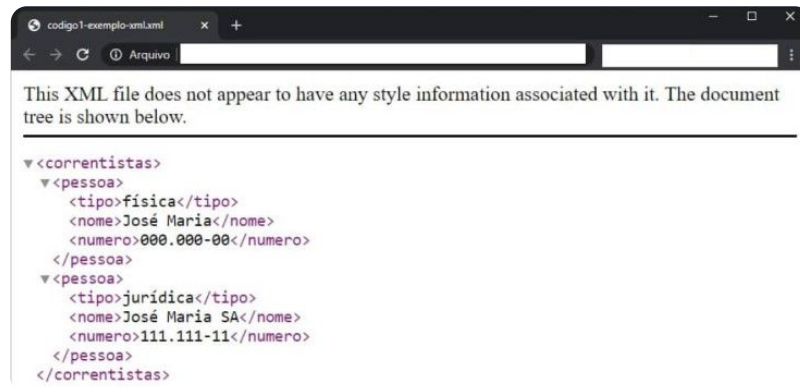
Em termos de sintaxe, observe os seguintes itens:

- Toda tag deve ser iniciada e fechada.
- Há um aninhamento representando hierarquia entre os dados. Por exemplo: as tags `<tipo>`, `<nome>` e `<numero>` estão indentadas, aninhadas e englobadas pela tag `< Pessoa>`. Da mesma maneira, todas as tags filhas estão aninhadas dentro do nó principal, `<correntistas>`.

Na imagem a seguir, veja como o documento XML é **renderizado** no navegador.

Renderizado

É o processo pelo qual se obtém o produto final de um processamento digital qualquer.



Documento XML exibido no navegador web.

Considerando sua sintaxe, dizemos que **XML** é um documento bem formatado. Logo, deixar de fechar uma tag ou construir um documento sem tag raiz torna o arquivo mal formatado ou inválido.

Na imagem a seguir, pode ser visto o resultado da renderização no navegador de um documento XML mal formatado – nesse caso, foi removida a tag de fechamento do elemento `< Pessoa>`, do documento XML visto anteriormente.



Renderização de XML mal formatado.

Utilizando atributos

Veja agora um novo exemplo de documento XML:

python

```
física
José Maria
000.000-00
```

```
jurídica
José Maria SA
111.111-11
```

Agora, veja sua renderização no browser:



Em relação ao primeiro documento XML, neste último, no elemento , foi adicionado o atributo “agencia” e um valor para esse atributo. Logo, podemos ter em um documento XML, além dos elementos, atributos. Inclusive, poderíamos modificar o documento anterior, trocando os elementos filhos de por atributos, gerando este novo documento:

python

Perceba que, no lugar de elementos filhos, os dados de cada conta foram armazenados diretamente no elemento , na forma de atributos. Sobre os atributos, como visto no exemplo, é necessário envolvê-los com aspas. Por último, veja que o elemento foi simplificado, sendo aberto e fechado em uma única linha, com a utilização da barra antes do sinal de maior (</>).

Em termos conceituais, um arquivo XML é composto pelos seguintes elementos:

1. Dados de caracteres (sequência de texto).
2. Instruções de processamento em anotações, normalmente inseridas no cabeçalho do documento.
3. Comentários, quando necessário.

4. Declaração de entidade.
5. Nós — elemento rotulado com um nome ou conjunto de atributos, cada um contendo nome e valor.

Por que utilizar XML?

Como vimos, ao tratarmos de sistemas web, a aplicabilidade da XML é servir como formato de transmissão de dados. Nesse sentido, considerando que esse formato foi especificado para ser simples, de fácil leitura por humanos e computadores, temos a principal justificativa quanto à sua adoção em sistemas web.

Além disso, podemos ainda mencionar as seguintes vantagens de utilização da XML:

- XML é autodocumentado, ou seja, seu formato descreve sua estrutura, seus elementos e valores.
- XML é independente de plataforma e linguagem de programação.
- XML é facilmente interpretado pela maioria das linguagens de programação (nas quais normalmente utilizamos recursos chamados de “parsers”, responsáveis por interpretar a estrutura e os dados do documento).
- XML é extensível. Logo, podemos tanto usar entidades criadas por outros quanto criar nossas próprias tags e atributos.
- XML possui suporte a Unicode.
- XML possui suporte à validação por meio de DTD (*Document Type Definition*) e Schema.



Saiba mais

Unicode é um padrão internacional que permite que todos os caracteres, de qualquer sistema de escrita existente, possam ser entendidos e manipulados por computadores. Schema é uma estrutura predefinida com regras de validação de um documento.

Convém também citar algumas desvantagens desse formato para a transmissão de dados por meio da internet em relação a outros disponíveis. Vamos conferir!

1

Desvantagem 1

Um documento XML possui a sintaxe detalhada e redundante quando comparada a outros formatos baseados em texto, como JSON.

2

Desvantagem 2

Um documento XML não suporta arrays.

3

Desvantagem 3

Um documento XML é menos legível que outros formatos, como JSON e YAML.

4 Desvantagem 4

Um documento XML pode gerar arquivos grandes dada a sua sintaxe detalhada.

XML e interface DOM

O DOM (Document Object Model) é uma interface de plataforma e linguagem neutra que permite que programas e scripts acessem e atualizem dinamicamente o conteúdo, a estrutura e o estilo de um documento (W3C, 2020).

Utilizando a interface DOM, é possível manipular tanto documentos HTML quanto documentos XML. Em ambos, o documento, por meio desse objeto, é representado na forma de árvore.

Manipulando um arquivo XML com a interface DOM

O XML DOM é um padrão para obter, modificar, adicionar ou remover elementos XML. Por meio dele, é possível acessar todos os elementos de um documento XML. Utilizando o documento XML já apresentado e, novamente, disponibilizado a seguir, veja os dois passos do código no qual, utilizando Javascript, o arquivo em questão é manipulado.

```
python
```

```
    física  
José Maria  
000.000-00
```

```
    jurídica  
José Maria SA  
111.111-11
```

```
python
```

Como visto no código, foi criada uma string JS que, em seguida, foi transformada em um documento XML válido. Estando nesse formato, foi possível manipular o seu conteúdo por meio do DOM. Com o método “getElementsByTagName” e as propriedades “childNodes” e “nodeValue”, foi acessado o conteúdo do primeiro elemento e atribuído o seu valor à tag HTML

, para exibição do resultado na tela.



Atenção

Nos exemplos aqui demonstrados, foi utilizada a linguagem de programação Javascript. Entretanto, conforme já mencionado, é possível usar qualquer linguagem de sua preferência para a manipulação do XML DOM.

Propriedades e métodos do XML DOM

Vejamos as propriedades do XML DOM:

- nodeName
- nodeValue
- parentNode
- childNodes
- attributes

E agora alguns dos métodos para o XML DOM disponíveis:

- getElementByTagName(name)
- getAttributeNode(node)
- appendChild(node)
- removeChild(node)
- createElement(name)
- createTextNode(value)

Outras formas de navegar por um documento XML

Existem outras formas de navegar pelos elementos e atributos de um documento XML, como:

XPATH

É uma recomendação do W3C e possui mais de 200 funções prontas para navegar em documentos XML utilizando a sintaxe “path like”.

XQUERY

É uma linguagem para realização de consultas no formato de queries – representando para o XML o mesmo que o SQL representa para os bancos de dados.

Recursos avançados

Além de tudo o que você aprendeu até aqui, há muito mais para se estudar e aprender a respeito de XML. Por exemplo: DTDs (Document Type Definition), schemas, entidades, notações, tipos de dados (datas, números, strings etc.). Embora o que vimos seja o bastante para utilizar o formato XML para a transmissão de dados em sistemas web, é recomendado que você conheça esses recursos mais avançados caso precise deles.



Saiba mais

Para maior aprofundamento no assunto, é recomendado que você pesquise sobre a especificação W3C.

Atividade

Questão 1

Em relação ao XML DOM, assinale a afirmativa correta.

A

A interface DOM relacionada a documentos XML nada tem a ver com a interface DOM relacionada a documentos HTML, já que ambos são linguagens não relacionadas.

B

Em XML, temos a liberdade de criar nossos próprios métodos dentro da interface DOM para manipulação de um documento, da mesma forma que temos liberdade para criar nossas próprias tags.

C

A interface DOM é um padrão para manusear documentos nos formatos HTML e XML. Por meio dessa interface, podemos acessar os elementos e atributos de um documento através das propriedades e dos métodos por ela disponibilizados.

D

A manipulação de documentos XML por meio do DOM só é possível com a linguagem de programação Javascript.

E

Por meio do padrão XML DOM é possível obter, adicionar ou remover elementos XML, entretanto não há como realizar modificações nos elementos acessados por esse padrão.



A alternativa C está correta.

O Document Object Model (DOM) é uma interface de plataforma e linguagem neutra. Logo, documentos escritos com linguagens de marcação como HTML e XML podem ser manuseados por meio dessa interface (sendo possível usar a maioria das linguagens de programação) que oferece métodos e propriedades distintos, para cada tipo de documento, que permitem acesso tanto aos seus elementos quanto ao seu conteúdo.

Criando o primeiro arquivo XML

Após conhecer o que é um arquivo XML e como ele pode ser utilizado, você agora criará um arquivo XML para transmissão de dados, conforme roteiro definido a seguir.

Neste vídeo prático, você aprenderá a criar o seu primeiro arquivo XML do zero! Passo a passo, iremos guiar você na criação de um documento XML válido, explicando a estrutura básica, a utilização de tags e atributos, e como organizar os dados. Você terá a oportunidade de colocar em prática todo o conhecimento adquirido sobre a linguagem de marcação XML, dando os primeiros passos na construção de arquivos XML para transmissão de dados em sistemas web. Não perca essa chance de se tornar expert na criação de XMLs!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

A prática, cujo roteiro será demonstrado logo a seguir, consistirá na criação e validação (em relação à estrutura) de um arquivo XML para transmissão de dados. Tanto os dados a serem inseridos como demais detalhes serão descritos na sequência.

Para realização do exercício, você precisará de:

- Um editor de textos – pode ser o próprio Bloco de Notas do Windows, o Nano Editor do Linux ou algo um pouquinho mais avançado, como o software (livre/gratuito) Notepad++,
- Um navegador para visualizar o XML – Google Chrome, Firefox, MS Edge etc.
- Uma ferramenta de validação de arquivos XMLs – existem diversas opções disponíveis on-line.

Vejamos agora o roteiro da nossa prática:

1

No editor, crie um arquivo XML para transmissão de dados referentes a uma nota fiscal eletrônica.

2

Em relação aos dados, a nota deverá ser composta por cinco seções:

- Cabeçalho
- Emitente
- Destinatário
- Item
- Rodapé

3

Cada uma das seções acima englobará dados, a saber:

- Cabeçalho: UF, descrição da operação, número de série, número da nota fiscal e data/hora de emissão da nota.
- Emitente: CNPJ, nome, nome fantasia e inscrição estadual.
- Destinatário: CNPJ, nome, e-mail e endereço. Esta última será uma subseção, composta pelos seguintes dados: logradouro, número, bairro, município, UF, CEP, país e telefone.
- Item: número do item (considerando que a nota poderá conter mais de um produto, cada item deverá ter um atributo chamado número, cujo valor é a ordenação da inclusão do produto na nota. Ex.: 1, 2, 3...), produto, valor e quantidade.
- Rodapé: valor total e valor do ICMS.

4

Após salvar o arquivo, com a extensão “.xml”, visualize-o no navegador.

5

Por fim, utilizando uma ferramenta on-line, valide o formato do arquivo, para garantir que tudo está certo.

Note a resolução do exercício no código seguinte:

python

RJ
Venda prod. do estab.
1
992346
2015-03-27T09:40:00-03:00

00822602000124
Plotag Sistemas e Suprimentos Ltda
Plotag - Localhost
114489114119

99999999000191
NF-E EMITIDA EM AMBIENTE DE HOMOLOGACAO - SEM VALOR FISCAL

Rua Jaragua
774
Bom Retiro
SAO PAULO
SP
01129000
BRASIL
33933501

email@yahoo.com.br

PAPEL MAXPLOT- 170MX250MX56GRS 3"
138.30
1

PAPEL MAX"
13.80
1

0.00
0.00

RJ
Venda prod. do estab.
1
992346
2015-03-27T09:40:00-03:00

00822602000124
Plotag Sistemas e Suprimentos Ltda
Plotag - Localhost
114489114119

99999999000191
NF-E EMITIDA EM AMBIENTE DE HOMOLOGACAO - SEM VALOR FISCAL

Rua Jaragua
774

Atividade

Um documento XML é formado por nós e atributos, sendo possível utilizar ambos para armazenar valores. Nesse contexto, analise o XML abaixo:

```
plain-text

  Tove
  Jani
  Reminder
  Don't forget me this weekend!
```

A partir desse XML inicial, altere sua formatação para que (apenas) a informação “heading” seja um atributo e não um nó. Acompanhe as opções a seguir.

I

```
plain-text

  Tove
  Jani
  Reminder
  Don't forget me this weekend!
```

II

```
plain-text

  Tove
  Jani

  Reminder

  Don't forget me this weekend!
```

III

```
plain-text

  Tove
  Jani
  Don't forget me this weekend!
```

IV

```
plain-text
```

V

```
plain-text
```

```
Tove  
Jani
```

```
Don't forget me this weekend!
```

```
Reminder
```

Assinale a alternativa cujo conteúdo corresponde, em termos de sintaxe, à alteração solicitada.

A

I.

B

II.

C

III.

D

IV.

E

V.



A alternativa C está correta.

O nó “heading” agora é um atributo do nó “body” e que mantém o seu valor original. As demais alternativas não atendem ao que foi solicitado, além de apresentarem problemas de sintaxe.

JSON e YAML como formatos de transmissão de dados

Por muitos anos, XML foi o principal formato para armazenamento e transmissão de dados na internet. Entretanto, após a sua criação, novos formatos foram e continuam sendo criados. Neste estudo, serão descritos dois formatos entre os mais utilizados — JSON e YAML. Com isso, após conhecer cada um desses três formatos, é esperado que você tenha os subsídios necessários para escolher, frente a cada cenário, qual formato utilizar no desenvolvimento de sistemas web.

Neste vídeo, exploraremos os formatos de transmissão de dados JSON e YAML. Abordaremos a sintaxe e estrutura de cada formato, bem como suas aplicações na transmissão de dados na web. Ao final, você terá o conhecimento necessário para escolher o formato adequado ao desenvolver sistemas web.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Formato JSON

JSON, acrônimo para Javascript Object Notation, é uma sintaxe para armazenamento e troca de dados. Trata-se de um formato de texto escrito com notação de objeto Javascript (W3C, 2020).



Exemplo

Com JSON, é possível representar dados de forma estruturada e transmiti-los na web, enviando e recebendo dados de servidores remotos e exibindo-os em páginas HTML ou em aplicativos.

A data de criação desse formato remete ao início dos anos 2000. Em 2001, ele foi apresentado pela primeira vez no site json.org. A especificação mais atual do JSON é a ECMA-404. Essa especificação define um pequeno conjunto de regras para a representação estruturada de dados e seu principal objetivo é definir a sintaxe de um documento JSON válido.

Sintaxe JSON

É composta por duas estruturas:

Coleção de pares "nome:valor"

Estrutura que, nas linguagens de programação, pode ser representada por objetos, dicionário, hash table, array associativo, entre outros.

Lista ordenada de valores

Nas linguagens de programação, pode ser representada como um array, vetor, lista, sequência, entre outros.

Apresentamos a seguir o JSON correspondente ao XML usado previamente. Após o código, cada elemento JSON será descrito.

```
python
{
    "correntistas": {
        "conta": [
            {
                "agencia": "0123-4",
                "tipo": "física",
                "nome": "José Maria",
                "numero": "000.000-00"
            },
            {
                "agencia": "5678-9",
                "tipo": "jurídica",
                "nome": "José Maria SA",
                "numero": "111.111-11"
            }
        ]
    }
}
```

O código visto também pode ser chamado de texto ou string JSON. Os dados após as chaves “agencia”, “tipo”, “nome” e “numero” são os valores JSON. O par chave: valor — como “agencia”: “0123-4” — é chamado de objeto JSON. A chave “conta” é um array. Vejamos esses elementos em detalhes!

Texto JSON

Um texto JSON é uma sequência de tokens formados a partir de código Unicode, em conformidade com a gramática JSON. Esse conjunto de tokens possui seis tokens estruturais: [(colchetes aberto),] (colchetes fechado), { (chaves aberta), } (chaves fechada), : (dois pontos) e , (vírgula).

Valores JSON

Um valor JSON pode ser: um objeto, um array, um número, uma string, true, false, null.

A estrutura de um objeto JSON é representada como um par de chaves “{” e “}” que engloba nenhum ou alguns pares de nome/valor, no qual o nome é uma string, seguido de dois pontos (:) que separam o nome do valor. Esse par “nome:valor” pode ou não ser procedido de um ou mais pares nome/valor, devendo esses serem separados por vírgula.



Dica

A sintaxe JSON não determina nenhuma restrição relacionada às strings utilizadas como nome, assim como não exige que os nomes sejam exclusivos. Além disso, a especificação JSON não define nenhum significado para a ordenação dos pares “nome:valor”.

Arrays em JSON

Um array, em JSON, é uma estrutura representada por colchetes que englobam nenhum ou alguns conjuntos de pares “nome:valor”, que deverão ser separados por vírgulas. Além disso, a sintaxe JSON não prevê

nenhuma forma para ordenar os valores do array. Como mencionado no exemplo anterior, a chave “conta” é um array.

JSON na prática

Vejam os, analisando alguns códigos, como enviar, receber e armazenar dados em JSON, utilizando a linguagem Javascript.

Enviando dados

Neste exemplo, é criado um objeto Javascript, que depois é convertido em texto JSON para ser enviado para um servidor remoto, onde poderá ser processado por uma linguagem server side, como Java, PHP etc.

```
python

//Cria um objeto Javascript
var objtoJS = {agencia: "5678-9", tipo: "física", nome: "Maria José", numero:
"222.222-22"};

//Converte o objeto Javascript em texto JSON
var textoJson = JSON.stringify(objtoJS);

//Redireciona a página para o endereço especificado, passando, via GET, o texto JSON
window.location = "processa/json/?conta=" + textoJson;
```

Recebendo dados

Neste exemplo, será criado um texto JSON, que será convertido em um objeto Javascript e então atribuído a um elemento HTML. Esse exemplo simula a situação em que, por meio de uma requisição AJAX, ou outro tipo de requisição, recebemos como retorno um texto JSON e precisamos manipulá-lo para acessar e utilizar seus dados.

```
python

//Representa um texto JSON (que poderia ter sido recebido de uma requisição, por ex)
var textoJson = '{"agencia": "5678-9", "tipo": "física", "nome": "Maria José",
"numero": "222.222-22"}';

//Converte o texto JSON em um objeto Javascript
var objtoJS = JSON.parse(textoJson);

//Exibe na tag html com id=resultado os dados da conta
document.getElementById("resultado").innerHTML = 'Agência: ' + objtoJS.agencia + ' Tipo:
' + objtoJS.tipo + ' Nome: ' + objtoJS.nome + ' Número: ' + objtoJS.numero;
```

Armazenando dados

O exemplo, a seguir, demonstra como armazenar dados no formato JSON. Para isso, será utilizado um objeto Javascript que será convertido em texto JSON e, em seguida, armazenado. Além disso, também será demonstrado como recuperar os dados armazenados.

python

```
//Cria um objeto Javascript
var objtoJS = {agencia: "5678-9", tipo: "física", nome: "Maria José", numero:
"222.222-22"};

//Converte o objeto Javascript em texto JSON
var textoJson = JSON.stringify(objtoJS);

//Armazenando os dados no navegador
localStorage.setItem("stringJSON", textoJson);
```

Para visualizar os dados armazenados no navegador com o método “localStorage”, após inserir o código acima em uma página HTML e executá-lo, com a página aberta no navegador, abra o inspecionador de elementos, navegue até a aba “Application” e, no menu à esquerda, opção “Storage”, vá até “Local Storage”. Nesse item, será possível ver as chaves e os respectivos valores armazenados (em nosso caso, a chave será “stringJSON”). Para recuperar os dados apresentados, armazenados com “localStorage”, utilize o código a seguir.

python

```
//Cria uma variável para receber o conteúdo armazenado com localStorage
var jsonText = localStorage.getItem("stringJSON");
//Converte em um objeto Javascript
jsOBJ = JSON.parse(jsonText);

//Exibe o conteúdo do objeto JS com o alert
alert('Agência: ' + jsOBJ.agencia + ' Tipo: ' + jsOBJ.tipo + ' Nome: ' + jsOBJ.nome + '
Número: ' + jsOBJ.numero);
```

YAML

YAML, acrônimo original para Yet Another Markup Language, e atualmente um acrônimo recursivo para YAML Ain't Mark-up Language, como o próprio nome diz em inglês, não é uma linguagem de marcação. Trata-se de uma linguagem para serialização e transmissão de dados cujo formato é amigável para humanos e de fácil entendimento para máquinas, podendo ser usada com a maioria das linguagens de programação.

Assim como XML e JSON, essa linguagem é usada para a transmissão de dados na internet.

Em comparação com os outros dois formatos, sua sintaxe é parecida, em termos de estrutura, com JSON. É comum vermos esse formato sendo utilizado como arquivo de configuração ou arquivo de logs, embora não fique limitada a essas funções.

Sintaxe YAML

Em termos de estrutura e sintaxe, a YAML tem por característica usar poucos caracteres estruturais a fim de permitir que os dados sejam exibidos de forma natural. Nesse sentido, a sintaxe de um arquivo YAML é composta por:

- Recuo/tabulação, usado como estrutura.
- Sinal de dois pontos (":"), usado como separador do par “chave: valor”.
- Travessão, usado para a criação de listas de marcadores.

Formato YAML

Podemos ver como YAML a mesma estrutura de dados já representada como XML e, posteriormente, como JSON:

```
python
```

```
correntistas:  
  conta:  
    - agencia: 0123-4  
      tipo: física  
      nome: José Maria  
      numero: 000.000-00  
    - agencia: 5678-9  
      tipo: jurídica  
      nome: José Maria SA  
      numero: 111.111-11
```

A fim de destacar as diferenças de estrutura, a seguir podem ser vistas as notações XML, JSON e YAML que representam a estrutura de dados que armazena dados de correntistas e que vem sendo utilizada como exemplo ao longo deste estudo.

Como podemos ver, YAML utiliza menos tokens, ou sinais, em sua estrutura, se comparada a XML e JSON — assim como o formato YAML privilegia a leitura e compreensão por humanos, em detrimento da interpretação por linguagens de programação.

YAML na prática

Veja, de forma prática, como trabalhar com dados armazenados com o formato YAML. No primeiro código, será den

Repare que, em ambos os exemplos, faz-se necessária a utilização de uma biblioteca de parser (a Standalone JavaScript question, por linguagem de programação, é mais complexo que o de outros formatos (como XML e JSON.).

```
python
```

Para executar o código, salve-o em uma pasta, em um servidor web, com o arquivo da biblioteca/parser e com o arquivo. Leia com atenção os comentários inseridos no código, pois explicam o que acontece linha a linha.

```
python
```

Todos os arquivos utilizados nesses exemplos podem ser encontrados na seção “Preparação”, dentro do arquivo compactado. Como vimos, se compararmos o processo de leitura e geração de conteúdo entre os três formatos descritos, veremos que a dificuldade para manuseio desses formatos pode variar. Logo, não existe um formato melhor do que o outro. Na verdade,

Atividade

Questão 1

Como vimos, o formato JSON possui uma sintaxe própria, formada por algumas características, sendo a principal a representação de uma string válida.

```
A
```

```
[3.14, true,"Joe"]
```

B

`["Value", 3.14, true, "name": "Joe"]`

C

`{"value": $3.14, "name": "Joe"}`

D

`{"name": "Joe", "age": 45, }`

E

`{"name": "Joe", "lastname": "Dude"s"" }`



A alternativa A está correta.

Na alternativa A, temos um array em formato Json, que representa uma string válida. Todas as demais possuem formato sobresaliente ao final da string, visto na opção D, ou pela utilização de aspas, sem que estas sejam "escapadas", dentro da string.

Criando o primeiro arquivo JSON

Assim como o XML, o JSON é um formato para transmissão de dados. Consequentemente, caso você precise realizar tal tarefa anteriormente, a seguir, você deverá criar e validar um arquivo JSON para a transmissão de dados seguindo o roteiro de prática.

Descubra, neste vídeo, como criar e validar um arquivo JSON eficiente para a transmissão de dados entre sistemas. Aprenda a utilizar o Notepad++ para a validação de arquivos JSON.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Neste exercício, você deverá criar e validar um arquivo no formato JSON, conforme o roteiro definido adiante. Você precisa utilizar o Notepad++ (gratuito) Notepad++. Caso opte por utilizar o Notepad++, esse software contém plugins para a visualização e navegação de arquivos JSON. Agora, vamos ao roteiro da prática?

1

No editor, crie um arquivo JSON para transmissão de dados referentes a uma nota fiscal eletrônica.

2

Em relação aos dados, a nota deverá ser composta por 5 “chaves-pai”:

- Cabeçalho
- Emitente
- Destinatário
- Item
- Rodapé

3

Cada uma das “chaves-pai” acima englobará dados, a saber:

- Cabeçalho: UF, descrição da operação, número de série, número da nota fiscal e data/hora de emissão da nota.
- Emitente: CNPJ, nome, nome fantasia e inscrição estadual.
- Destinatário: CNPJ, nome, email e endereço. Esta última será uma subseção, composta pelos seguintes dados:
- Item: número do item (considerando que a nota poderá conter mais de um produto, cada item deverá der um produto).
- Rodapé: valor total e valor do ICMS.

4

Após salvar o arquivo, com a extensão “.json”, visualize/navegue pelo mesmo utilizando o plugin do editor Notepad++.

5

Por fim, utilizando uma ferramenta on-line, valide o formato do arquivo para garantir que tudo está certo.

Agora, veja o código dessa atividade!

python

```
{
  "NFe": {
    "CABECALHO": {
      "UF": "RJ",
      "OPERACAO": "Venda prod. do estab.",
      "SERIE": 1,
      "NF": 992346,
      "DTHREMISSAO": "2015-03-27T09:40:00-03:00"
    },
    "EMITENTE": {
      "CNPJ": 822602000124,
      "NOME": "Plotag Sistemas e Suprimentos Ltda",
      "NOMEFANTASIA": "Plotag - Localhost",
      "INCEST": 114489114119
    },
    "DESTINATARIO": {
      "CNPJ": 99999999000191,
      "NOME": "NF-E EMITIDA EM AMBIENTE DE HOMOLOGACAO - SEM VALOR FISCAL",
      "ENDERECO": {
        "LOGRADOURO": "Rua Jaragua",
        "NRO": 774,
        "BAIRRO": "Bom Retiro",
        "MUNICIPIO": "SAO PAULO",
        "UF": "SP",
        "CEP": 1129000,
        "PAIS": "BRASIL",
        "FONE": 33933501
      },
      "EMAIL": "email@yahoo.com.br"
    },
    "ITEM": [
      {
        "PRODUTO": "PAPEL MAXPLOT- 170MX250MX56GRS 3\\\"",
        "VALOR": 138.3,
        "QUANTIDADE": 1
      },
      {
        "PRODUTO": "PAPEL MAX\\\"",
        "VALOR": 13.8,
        "QUANTIDADE": 1
      }
    ],
    "RODAPE": {
      "VALORTOTAL": 0,
      "ICMS": 0
    }
  }
}
```

Atividade

Questão 1

Como vimos, cada formato de transmissão de dados possui sua própria sintaxe. A seguir, apresentaremos um documento

```
python
```

```
NFe:
```

```
  CABECALHO
```

```
    UF: RJ
```

```
    OPERACAO: Venda prod. do estab.
```

```
    SERIE: 1
```

```
    NF: 992346
```

```
    DTHREMISSAO: '2015-03-27T09:40:00-03:00'
```

```
  EMITENTE
```

```
    CNPJ: 822602000124
```

```
    NOME: Plotag Sistemas e Suprimentos Ltda
```

```
    NOMEFANTASIA: Plotag - Localhost
```

```
    INCEST: 114489114119
```

```
  DESTINATARIO
```

```
    CNPJ: 99999999000191
```

```
    NOME: NF-E EMITIDA EM AMBIENTE DE HOMOLOGACAO - SEM VALOR FISCAL
```

```
  ENDERECO:
```

```
    LOGRADOURO: Rua Jaragua
```

```
    NRO: 774
```

```
    BAIRRO: Bom Retiro
```

```
    MUNICIPIO: SAO PAULO
```

```
    UF: SP
```

```
    CEP: 1129000
```

```
    PAIS: BRASIL
```

```
    FONE: 33933501
```

```
    EMAIL: email@yahoo.com.br
```

```
  ITEM
```

```
- PRODUTO: PAPEL MAXPLOT- 170MX250MX56GRS 3"
```

```
  VALOR: 138.3
```

```
  QUANTIDADE: 1
```

```
- PRODUTO: PAPEL MAX"
```

```
  VALOR: 13.8
```

```
  QUANTIDADE: 1
```

```
  RODAPE
```

```
    VALORTOTAL: 0
```

```
    ICMS: 0
```

A

Um arquivo YAML não pode conter chaves escritas em letras maiúsculas.

B

Um arquivo YAML não pode ter uma chave principal – no caso do exemplo, a chave “NFe”.

C

Para ser vá lido, um arquivo YAML deverá conter uma chave principal aberta em seu início e fechada ao seu final. Logo,

D

Uma das principais características, e também vantagem de um arquivo YAML, é o seu tamanho reduzido. Logo, para ma

E

O arquivo está inválido pois as seções principais não estão terminadas com o símbolo “:” (dois pontos).



A alternativa E está correta.

Arquivos YAML possuem uma estrutura de chaves seguidas de valores ou de outras chaves (seguidas de valores). A seção "pai".

Utilizando os formatos de transmissão de dados com AJAX

Após conhecer alguns dos formatos de transmissão de dados, chegou a hora de utilizá-los na prática. Nesse sentido, veja o vídeo.

Neste vídeo, você aprenderá sobre os formatos de transmissão de dados em requisições AJAX, com foco nos formatos XML e JSON.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

XML e JSON em requisições AJAX

Inicialmente, veremos como utilizar dados armazenados no formato XML por meio de requisições AJAX. Embora aqui se fale em JSON, desde que, claro, estejam no formato em questão. Para isso, troque o endereço do arquivo local para o do recurso em questão.



Atenção

Demonstraremos apenas o primeiro nó para entendimento da estrutura. Os demais dados estarão no arquivo disponível no link.

XML e AJAX

Vamos ao código, começando com o arquivo XML, lido a seguir pelo arquivo HTML e Javascript, que consumirá os dados.

```
python
```

```
7631
HEH-9133
On Cloud Nine Pillow
https://www.domain.com/product/heh-9133
24.99
24.99
https://www.domain.com/images/heh-9133_600x600.png
lorem, ipsum, dolor, ...
Sociosqu facilisis duis ...
Home>Home Decor>Pillows|Back In Stock
298|511
FabDecor
```

```
White
White
```

```
4.2
8
21
2018-03-03 17:41:13
```

Ao analisar o código apresentado previamente, percebe-se que é realizada uma verificação em relação ao tipo de nó, no p

```
python
```

```
noFilho.nodeType === 1
```

Essa verificação é necessária, uma vez que os espaços em branco, no arquivo XML, também são considerados como nó

» #t
» pr
» #t
» sk
» #t
» na
» #t
» pr
» #t
» pr
» #t
» re
» #t
» th
» #t
» se
» #t
» de
» #t
» ca
» #t
» ca
» #t

Fragment

JSON e AJAX

Veremos agora como trabalhar com JSON e AJAX. Nesse contexto, o primeiro código contém um fragmento do arquivo

python

```
{
  "Products": {
    "Product": [
      {
        "Product_ID": "7631",
        "SKU": "HEH-9133",
        "Name": "On Cloud Nine Pillow",
        "Product_URL": "https://www.domain.com/product/heh-9133",
        "Price": "24.99",
        "Retail_Price": "24.99",
        "Thumbnail_URL": "https://www.domain.com/images/heh-9133_600x600.png",
        "Search_Keywords": "lorem, ipsum, dolor, ...",
        "Description": "Sociosqu facilisis duis ...",
        "Category": "Home>Home Decor>Pillows|Back In Stock",
        "Category_ID": "298|511",
        "Brand": "FabDecor",
        "Child_SKU": "",
        "Child_Price": "",
        "Color": "White",
        "Color_Family": "White",
        "Color_Swatches": "",
        "Size": "",
        "Shoe_Size": "",
        "Pants_Size": "",
        "Occassion": "",
        "Season": "",
        "Badges": "",
        "Rating_Avg": "4.2",
        "Rating_Count": "8",
        "Inventory_Count": "21",
        "Date_Created": "2018-03-03 17:41:13"
      }
    ]
  }
}
```

Como visto, estamos trabalhando com a mesma estrutura de dados usada no primeiro exemplo, em XML, ou seja, uma l

python

A exemplo do que foi feito com o arquivo XML, o código acima trata os dados recebidos da requisição AJAX — aqui no f
algumas de suas partes.

Principais diferenças no processamento de XML e JSON

Os códigos vistos tratam da requisição e manipulação dos dados nos formatos XML e JSON e contêm alguns comentários para explicar as diferenças!

responseXML x responseTEXT

Por meio de requisições AJAX, é possível recuperar dados de recursos remotos em diferentes formatos, como XML e responseXML.

Com o `responseText`, tratamos os dados recebidos que não estejam no formato XML. Logo, teremos os dados da resposta em um objeto JS. Com isso, conseguimos acessar cada par “chave: valor” do arquivo JSON na notação de objeto. Com o `responseText`

Dados como objetos JS

Quando estiver trabalhando com AJAX, e sempre que possível, converta o resultado obtido em objeto JS. Isso facilita

Uma dica para descobrir qual o formato dos dados recebidos é utilizar o método “console.dir” no resultado da requisição. Assim, seguindo o exemplo anterior, basta substituir o `responseXML` por `console.dir(responseXML)` e o `responseText` por `console.dir(responseText)`. Assim, seguindo o exemplo anterior, basta substituir o `responseXML` por `console.dir(responseXML)` e o `responseText` por `console.dir(responseText)`. Além disso, é possível utilizar o método `console.log` para imprimir o resultado da requisição no console do navegador.

Nas próximas imagens, é possível reparar onde apenas parte dos resultados da aplicação do método "console.dir" sobre

Nesta primeira imagem, veja a saída no formato XML, em que, já na primeira linha, temos a informação “#document”. Em
relacionadas especificamente ao formato em questão.

```
▼ #document
  URL: "http:
  activeElemen
  ▶ adoptedStyl
  ▶ alinkColor:
  ▶ all: HTMLAI
  ▶ description
  ▶ anchors: HT
  ▶ applets: HT
  ▶ baseURI: "h
  ▶ bgColor: ""
  ▶ body: null
  ▶ characterSe
  ▶ charset: "U
  ▶ childElemen
  ▶ childNodes:
  ▶ children: H
  ▶ compatMode:
  ▶ contentType
  ▶ cookie: ""
  ▶ currentScri
  ▶ defaultView
  ▶ designMode:
  ▶ dir: ""
  ▶ doctype: nu
  ▶ documentEle
```

Primeira

Agora, na próxima imagem, temos apenas um texto plano na string JSON:

```
{  
  "Pro  
  "I
```

Segunda

Por fim, observe na última imagem que após aplicação do “JSON.parser” sobre a string JSON, na primeira linha, temos o

```
ay(20)  
t_ID: "7631",  
t_ID: "7615",  
t_ID: "8100",  
t_ID: "6489",  
t_ID: "7732",  
t_ID: "7609",  
t_ID: "7675",  
t_ID: "7463",  
t_ID: "7677",  
t_ID: "8099",  
ct_ID: "7425",  
ct_ID: "8102",  
ct_ID: "9964",  
ct_ID: "10440",  
ct_ID: "6060",  
ct_ID: "10448",  
ct_ID: "8137",  
ct_ID: "10018",  
ct_ID: "5670",  
ct_ID: "9020",  
  
Array(0)  
Object  
Object
```

Terceira p

YAML e AJAX

O trabalho com dados transferidos no formato YAML é semelhante ao que vimos em relação ao JSON, uma vez que os dados são transferidos no formato de string. Nesse caso, um passo adicional, que é o de transformá-los no formato de objeto JS/JSON.



Recomendação

Para essa tarefa, utilize um parser, uma biblioteca de terceiros. Após a conversão, todo o código restante será exatamente o mesmo que os exemplos repetidos aqui.

Por fim, podemos ver o fragmento do arquivo YAML contendo os dados dos produtos, trata-se do mesmo conteúdo dos

```
python
```

```
Products:
```

```
Product:
```

```
- Product_ID: '7631'
```

```
SKU: HEH-9133
```

```
Name: On Cloud Nine Pillow
```

```
Product_URL: https://www.domain.com/product/heh-9133
```

```
Price: '24.99'
```

```
Retail_Price: '24.99'
```

```
Thumbnail_URL: https://www.domain.com/images/heh-9133_600x600.png
```

```
Search_Keywords: lorem, ipsum, dolor, ...
```

```
Description: Sociosqu facilisis dui...
```

```
Category: Home>Home Decor>Pillows|Back In Stock
```

```
Category_ID: 298|511
```

```
Brand: FabDecor
```

```
Child_SKU: ''
```

```
Child_Price: ''
```

```
Color: White
```

```
Color_Family: White
```

```
Color_Swatches: ''
```

```
Size: ''
```

```
Shoe_Size: ''
```

```
Pants_Size: ''
```

```
Occasion: ''
```

```
Season: ''
```

```
Badges: ''
```

```
Rating_Avg: '4.2'
```

```
Rating_Count: '8'
```

```
Inventory_Count: '21'
```

```
Date_Created: '2018-03-03 17:41:13'
```

Atividade

Questão 1

Uma requisição AJAX pode retornar dados em diferentes formatos. Nesse sentido, é correto afirmar que:

A

Dados de diferentes tipos podem ser tratados por meio dos métodos `responseXML` e `responseText`, sendo o primeiro responsável pela manipulação.

B

Há vários métodos disponíveis para o tratamento de dados, conforme o seu formato, como o `responseXML`, o `responseText`, o `responseJSON`, entre outros.

C

Em linguagens como o Javascript, todos os dados retornados a partir de uma requisição AJAX, independentemente do formato, são tratados como texto.

D

Não é possível, apenas utilizando recursos nativos da linguagem Javascript, manipular dados em diferentes formatos. Para isso, é necessário utilizar bibliotecas externas.

E

Os resultados obtidos por uma requisição AJAX podem ser transformados em objeto XML. Essa prática facilita a tarefa de



A alternativa A está correta.

As requisições AJAX retornam dados em dois formatos: XML e Texto. Logo, qualquer formato que não o XML é visto como texto. Se o formato XML é utilizado, seja por meio de recursos nativos da própria linguagem seja por bibliotecas de terceiros.

Consumindo dados no formato JSON com AJAX

Atualmente, o formato mais utilizado para a transmissão de dados no ambiente web é o JSON. Para consumir dados JSON com Javascript. Nesse contexto, ao longo dessa prática, usaremos cada uma dessas tecnologias para requisitar, consumir, manipular e exibir os dados.

Neste vídeo, exploraremos o formato JSON e como utilizá-lo em requisições AJAX. Aprenderemos a consumir dados JSON com Javascript.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Para realização dessa prática, você precisará de alguns recursos. Vejamos!

- Inicialmente, será necessário um editor para construir seu código-fonte. Você poderá usar o próprio Bloco de Notas ou qualquer outro editor de texto.
- Também será necessário um navegador web, como o Google Chrome, Firefox, MS Edge etc., para exibir a página.
- Por fim, precisará do endereço de uma API REST que seja gratuita e retorne dados em formato JSON. Sugerimos a API <https://swapi.dev/>, onde você poderá utilizar qualquer outra API (lembrando apenas que o formato devolvido por ela deverá ser JSON).

O fluxo da página HTML será o seguinte:

1

No carregamento da página, deverá ser feita uma requisição AJAX à API e coletados alguns dados, como os de um personagem. Para isso, usaremos o URI `https://swapi.dev/api/people/1/`, onde o número ao final representa o seu código, havendo alguns disponíveis na API.

2

De acordo com a API escolhida ou o recurso utilizado, serão devolvidos diferentes dados no JSON. Logo, você precisará manipular esses dados para exibir a página.

3

Em seguida, esses dados coletados deverão ser inseridos em uma estrutura HTML para sua exibição. Aqui você poderá usar o `innerHTML` para inserir o HTML na página.

Agora, vejamos o roteiro propriamente dito:

- No editor, crie o esqueleto de uma página HTML básica.
- Dentro da tag `body`, crie uma `div` e atribua um `id` para a mesma. Essa `div` será usada para exibir os dados obtidos da API.
- Dentro do arquivo, na seção `header`, crie o código JS para realizar a requisição AJAX, via `xmlHttpRequest`, da API.

- Ainda no código JS, crie uma função para receber o resultado da requisição AJAX.
- Na função criada, você deverá utilizar o objeto contendo a resposta (responseText) para ter acesso ao JSON.
- Faça o parse do JSON em um objeto Javascript para facilitar o acesso aos dados recebidos.
- Estando com acesso, via objeto JS, a cada dado recebido, utilize-os para montar a estrutura HTML necessária p
- Após salvar o arquivo, com a extensão “.html”, visualize a página utilizando o navegador de sua preferência.
- Tendo ocorrido tudo certo, você deverá ver na página os dados coletados, organizados na estrutura HTML que v
- Caso nenhum dado seja exibido, cheque a aba “console”, no inspecionador de elementos, no navegador, e veja s

A resolução da prática acima poderá ser feita de diferentes formas. Veja, a seguir, uma das formas possíveis.

```
python
```

Atividade

Questão 1

A estrutura de um arquivo XML é composta por nós “pais” e nós “filhos”. Logo, é importante entender essa sintaxe para XML. Assinale qual a afirmativa, entre as opções abaixo, equivale à saída do código, na instrução “alert”.

```
python
```

A

“multiples”

B

“Tove”

C

“undefined”

D

“multiples” e “Tove”

E

“Jani”



A alternativa B está correta.

Para manipular dados no formato XML, podemos fazer uso da XML DOM ou de outros recursos, como o XPATH. Em relação ao mesmo nome, "to". Entretanto, a forma de acessá-los é diferente: para o primeiro, temos o "getElementsByTagName" e para o segundo, temos o "getElementsByXPath", sobretudo ao fazer uso de índices para acessar os nós do documento.

Considerações finais

Você aprendeu sobre os conceitos e linguagens:

- Formatos de transmissão de dados em sistemas web
- Diferentes formatos de dados em requisições AJAX
- XML
- JSON
- YAML

Explore +

Para saber mais sobre os assuntos tratados neste estudo, pesquise na internet:

Arrow Functions: A especificação Javascript ES6 apresentou alguns importantes novos recursos. Embora não possua suporta as arrow functions. Veja como o guia de referência Javascript da Fundação Mozilla aborda as arrow functions, introduzindo

API REST: As APIs estão intimamente ligadas aos conceitos vistos em nosso estudo. Para saber mais a seu respeito, recomendamos

- O tutorial **JSON with Ajax**, publicado no site REST API Tutorial em 17 de agosto de 2021.
- O site do JSON API.
- O artigo **O que é API REST?**, publicado no site Red Hato em 8 de maio de 2020.

Referências

EXTENSIBLE Markup Language (XML). W3C schools. Consultado na internet em: 20 ago. 2020.

JSON Introduction. W3C schools. Consultado na internet em: 20 ago. 2020.

XML DOM Tutorial. W3C schools. Consultado na internet em: 20 ago. 2020.